

从轻量底座到团队级智能体运行框架：pi agent 定制实战

SSS & Codex

2026 年 7 月 26 日



视频作者/频道：程序员暮闲

发布日期：2026 年 6 月 6 日

视频时长：31 分 55 秒

视频链接：<https://www.bilibili.com/video/BV1ZnEJ6NEJ6/>

目录

1 先做选择：pi agent 适合哪类 Harness Agent	3
1.1 选择题先于安装	3
1.2 pi agent 提供的基础与留出的空间	4
1.3 能力阶梯总览	5
1.4 本章小结	6
2 从空白底座到首个可调用工具	6
2.1 最小可用链路	6
2.2 扩展的基本结构与加载作用域	7
2.3 change_context：从 Git 到模型	9
2.4 验证闭环：加载提示只是第一项证据	9
2.5 本章小结	10
3 把运行时变成可观察、可拦截、可验证的系统	10
3.1 生命周期事件是运行时挂点	11
3.2 危险动作确认闭环	12
3.3 斜杠命令是可验证的运行入口	12
3.4 自定义模型供应商先完成注册	13
3.5 四个状态必须分别验证	14
3.6 可视化界面必须暴露持久化状态	16
3.7 本章小结	16
4 让一次演示变成可复用任务：技能与提示模板	16
4.1 渐进加载控制任务说明的读取时机	16
4.2 四种来源决定复用范围	17
4.3 change-assessment 固化评估方法	18
4.4 提示模板把方法封装成稳定入口	19
4.5 三层案例：数据、方法与入口各司其职	20
4.6 何时写扩展、何时写技能、何时写提示模板	21
4.7 本章小结	22
5 从个人配置到团队交付：主题与 pi 扩展包	22
5.1 主题的能力边界：统一颜色表达	22
5.1.1 完整 JSON 是配置合同	22
5.2 pi 扩展包：把能力资产变成分发单元	23
5.2.1 目录表达内容，清单表达意图	24
5.3 安装之后还要证明资源已加载	24
5.4 从演示包走向团队治理	25
5.5 本章小结	26

6 总结与延伸：一条可落地的定制路线	26
6.1 讲者的最终判断：先确定是否需要定制	26
6.2 六步实施路线：从边界选择到团队核验	27
6.2.1 第一步：明确要控制的边界	27
6.2.2 第二步：接通最小模型链路	27
6.2.3 第三步：先做一个可验证工具	28
6.2.4 第四步：加入生命周期与安全策略	28
6.2.5 第五步：用技能和提示模板固定任务	28
6.2.6 第六步：用 pi 扩展包分发并做安装后核验	28
6.3 两个闭环，共同证明一条工程原则	29
6.3.1 代码变更评估闭环	29
6.3.2 自定义供应商路由闭环	29
6.4 限制、风险与未知项	29
6.5 从个人定制走向可信团队运行	30
6.5.1 版本治理与可信发布	30
6.5.2 从字符串规则升级为结构化策略	30
6.5.3 多供应商环境中的自动审计	30
6.6 最终记忆卡	31
6.7 本章小结	31

1 先做选择：pi agent 适合哪类 Harness Agent

pi agent 的适用价值可以先压缩成一句话：当开发者需要掌控工具、运行事件、安全策略、模型接入和团队资产时，这个轻量底座提供了清晰的扩展边界；当目标是安装后立即获得一套完整能力时，视频建议优先评估预装型方案。这个判断应当发生在安装之前，因为轻量同时意味着更大的设计自由和持续的工程责任。

本章核心判断

pi agent 适合愿意自行组合能力、验证运行结果并承担维护责任的定制开发者。产品选择的关键问题是“需要控制哪些边界”，功能名称的多少无法替代这项判断。

1.1 选择题先于安装

智能体运行框架（Harness Agent）是围绕模型连接工具、上下文、生命周期和交互界面的运行外壳。它的范围大于一次模型对话：模型负责推理与生成，运行框架负责把模型放进一个能够读取环境、调用工具、保持约束并向用户反馈的系统中。

视频用两类目标划定了选择边界。第一类目标强调即时使用，希望安装后便获得记忆、交流通道、多智能体协作等上层能力；讲者把 OpenClaw、Hermes 作为这类预装型方案的例子。第二类目标强调控制，希望自行决定工具如何注册、事件何时触发、危险动作怎样确认、模型如何接入，以及团队资源怎样分发。pi agent 面向第二类目标提供更直接的组合空间。

表 1: 从目标出发的产品选择矩阵

判断维度	即时可用优先	定制控制优先
首要诉求	尽快取得已经组合好的上层能力	自行定义工具、事件、安全和模型接入
初始投入	主要投入在安装、配置与学习既有用法	还需投入设计、实现、测试和维护
运行边界	由产品既有设计提供	由扩展接口和项目规范共同形成
视频中的选择提示	可评估 OpenClaw 或 Hermes	pi agent 更匹配这一目标

这张矩阵只复述视频证明的定位。它没有为 OpenClaw、Hermes 建立完整功能清单，也没有证明 pi agent 内置了完整记忆、交流通道或多智能体系统。视频中的主张是：这些能力可以沿公开接口继续开发；接口可能性仍需具体实现和运行证据才能转化为可交付能力。

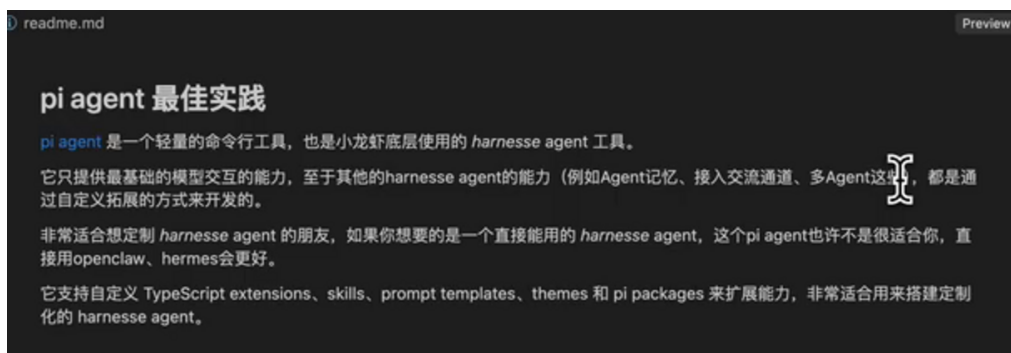


图 1: 视频 README 把 pi agent 定位为轻量命令行底座，同时列出五类扩展面。画面中的产品边界与前述选择矩阵相互印证。

1

轻量底座的隐含成本

上层能力需要开发者自行设计、测试和维护。工具失败怎样暴露、凭据如何保护、危险动作如何授权、扩展如何升级，都属于落地责任。视频展示的是教学级组合能力，未证明生产级权限系统、审计系统或多智能体编排已经随产品交付。

1.2 pi agent 提供的基础与留出的空间

pi agent 默认把重心放在命令行中的模型交互，并把更多行为留给扩展（extension）。这里的“扩展”专指通过 TypeScript 接口加入运行时工具、事件处理器、命令或模型接入的代码单元；它和后文保存于 SKILL.md 的任务说明承担不同职责。

可以把这个边界理解为“底座先规定插座，开发者再选择接入什么设备”。底座提供可加载的接口和目录约定，扩展负责把实际行为接进去。视频展示的扩展面包括：

- TypeScript 扩展：注册工具、监听运行事件、介入工具调用、注册斜杠命令和模型接入；
- 任务方法资源：保存可复用的方法与输出要求；
- 调用编排资源：形成稳定入口，并进一步编排任务方法与工具；
- 主题（theme）：调整终端输入、输出、思考文本、分隔线和边框等颜色；
- 团队分发资源：把一种或多种资源组织成可安装、可核验的单元。

接口扩展和修改源码的边界也很清楚。工具、任务方法、模型接入或颜色配置可以沿公开扩展面完成。只有当目标涉及 pi agent 自身整体交互形态，现有接口无法表达所需变化时，源码修改才成为候选路线。这个顺序能够减少与底座实现的耦合，也让团队更容易独立升级自己的能力资产。

TypeScript 经验不是进入门槛

视频提醒开发者：运行时扩展使用 TypeScript 编写。读者仍需理解参数、异步执行、错误传播和安全边界；语言本身可以借助智能体学习和生成。工程验收依赖代码审查与运行验证，语言生成速度无法替代这些检查。

¹ 视频来源时间：00:00:28.870–00:01:04.680；所选稳定帧：00:00:47。

1.3 能力阶梯总览

五类扩展面可以整理为四层能力阶梯，并把主题视为横向外观层。每一层回答的问题不同：

1. **运行时层**：扩展和工具决定系统能够观察什么、执行什么、拦截什么；
2. **任务方法层**：技能（Skill）通过独立目录和 `SKILL.md` 固化某类任务的方法与输出要求；
3. **调用编排层**：提示模板（prompt template）把技能与工具组织成稳定入口，并可形成斜杠命令；
4. **团队分发层**：pi 扩展包（pi package）通过 `package.json` 中的清单组织并分发多类资源。

主题横跨这些层，负责终端颜色契约。视频只证明了颜色相关配置，没有证明主题能够重新设计交互结构。pi 扩展包也不要求同时包含所有资源类型：一个包可以只包含扩展，也可以组合扩展、技能、提示模板和主题。

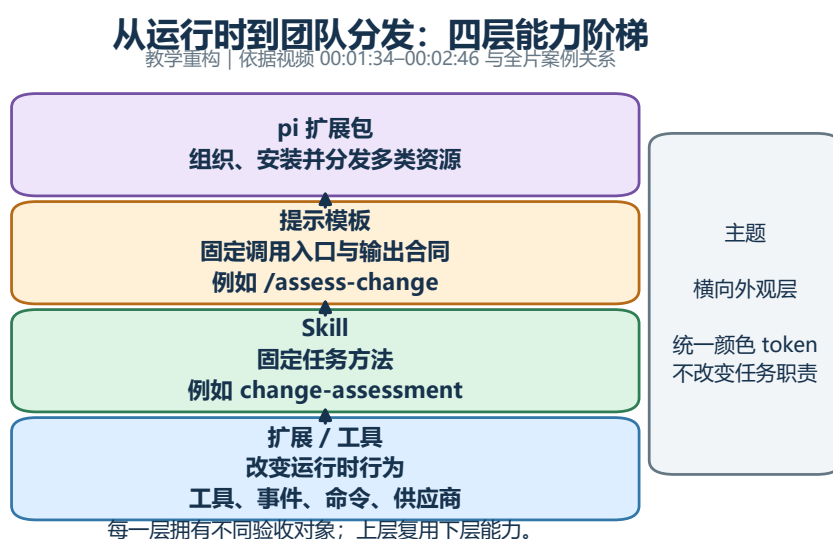


图 2：根据视频扩展分类制作的教学重构。纵向层级把运行时工具、`change-assessment` 技能、`/assess-change` 提示入口和 pi 扩展包放进同一案例链；主题只承担画面确认的颜色外观职责。

2

贯穿全文的“代码变更影响评估”案例可以提前放进这套阶梯中理解：

数据与动作 `change_context` 工具读取 Git 变更并返回文件列表与 diff；

任务方法 `change-assessment` 技能规定怎样判断改动、影响、风险和复核建议；

稳定入口 `/assess-change` 提示命令固定调用顺序和输出结构；

运行约束 安全扩展在危险工具调用前加入确认与阻塞；

团队交付 pi 扩展包把这些资产交给其他成员安装和核验。

这一分层避免把“工具能执行”误当成“任务已经工程化”。工具提供能力，技能提供方法，提示模板提供入口与格式，pi 扩展包提供分发契约。只有各层分别验证，团队才能判断一个能力究竟卡在数据采集、任务方法、调用入口还是安装加载。

²内容来源时间：00:01:34.030-00:02:46.120；本图为基于视频分类关系的教学重绘。

1.4 本章小结

pi agent 的选择可以归结为三问：是否需要自定义工具，是否需要控制生命周期与安全，是否需要统一团队资产。三问中有明确的控制需求时，轻量底座提供了有价值的扩展空间；目标集中于立即使用完整能力时，预装型方案更贴近视频给出的选择建议。

下一章将沿能力阶梯进入第一层：先接通最小模型链路，再把一个 TypeScript 扩展从文件定义推进到模型实际调用和结构化结果验证。

2 从空白底座到首个可调用工具

安装和登录只建立模型连通性。一个真正完整的扩展闭环还需要完成定义、发现、重载、模型选择调用和结果核验。本章承接图 2 的运行时层，以视频中的 `change_context` 为例，展示如何把 Git 工作区变更变成模型可使用的结构化上下文。

本章核心判断

“配置完成”和“运行生效”是两个状态。扩展文件存在、启动界面显示已加载、模型实际选择工具、工具返回预期结果，需要分别取得证据。

2.1 最小可用链路

视频演示的最小链路包含四步：全局安装、检查版本、在目标项目启动、通过 `/login` 接入模型。画面能够确认的安装命令如下：

```
1 npm install -g --ignore-scripts @earendil-works/pi-coding-agent
2 pi --version
3 pi
```

Listing 1: 视频画面确认的安装与启动入口

`pi --version` 用于确认命令已经可用；在目标项目目录执行 `pi` 后，首次启动可能提示安装缺失的 `fd`。随后通过 `/login` 选择模型供应商（model provider）和凭据方式。模型供应商提供接口地址、认证契约、API 适配和模型清单；供应商出现在列表中，只说明它可以被选择和配置。

演示画面中的 `pi` 版本为 `v0.78.1`，并显示 33 个供应商选项。这两个数字都属于当时版本的界面证据，不能被解释为长期固定承诺。读者应依据当前环境选择实际可用的供应商，也应通过普通对话确认模型链路已经接通。

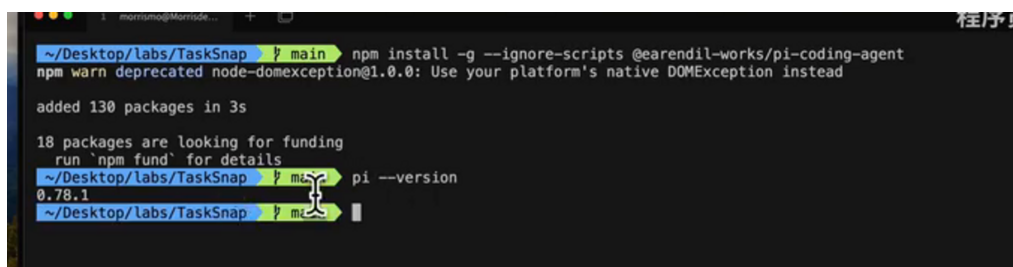


图 3: 安装命令完成后，终端用 `pi --version` 返回 `v0.78.1`。裁剪保留命令、安装结果和版本输出，用于证明“安装—验证”两步均有独立证据。

版本命令证明可执行文件存在，启动界面还需要证明模型状态。下面的上下两帧分别保留首次启动的缺少模型提示和 `/login` 后的供应商选择列表，未包含凭据创建、粘贴或输入画面。

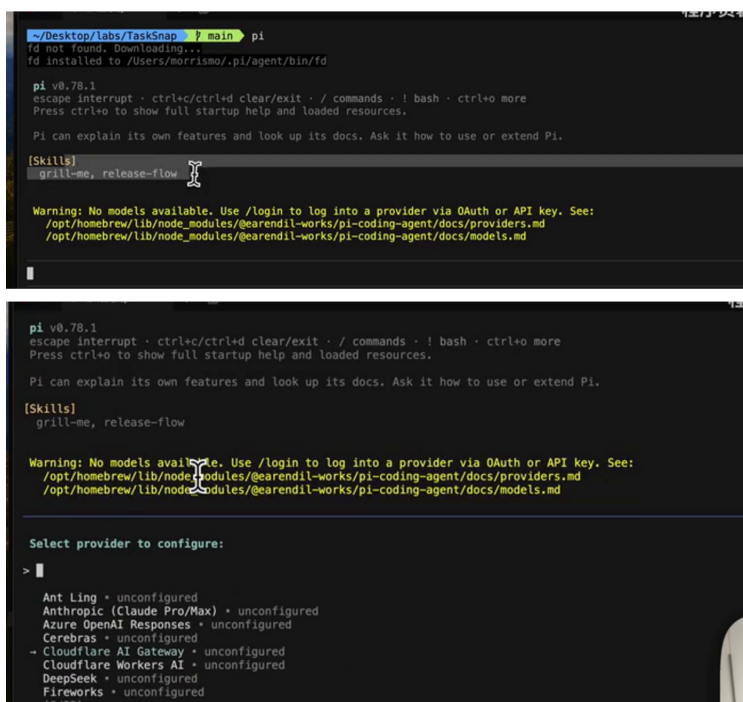


图 4: 首次启动与供应商选择的两个状态。上图明确要求通过 `/login` 接入供应商；下图显示供应商列表已经打开。两帧共同说明“底座启动”和“模型接入”需要分步完成。

4

凭据边界

API key 应通过受控输入或环境配置进入认证流程。正文、截图、日志、命令历史和团队交接都不应复现实际密钥。视频中的具体输入仅服务于演示，无法成为安全配置模板。

此时的系统只完成了“模型可交互”。视频用“白纸”形容启动后的状态：界面显示的技能来自当前项目，更多工具和任务能力仍需沿扩展接口加入。这个状态恰好解释了第一章的选择原则——轻量底座把能力设计权和维护责任交给使用者。

2.2 扩展的基本结构与加载作用域

运行时扩展的最小外形是一个 TypeScript 文件，其默认导出函数接收 `ExtensionAPI`。函数内部通过 `pi.registerTool(...)` 等接口注册行为。下面的片段只保留画面与分析报告能够确认的结构，省略小字号、未核清的字段：

```
1 export default function (pi: ExtensionAPI) {
2   pi.registerTool({
3     name: "change_context",
4     // description, parameters, execute ...
5   });
```

³视频来源时间：00:03:28.950–00:03:57.410；所选稳定帧：00:03:54。

⁴视频来源时间：00:04:04.620–00:04:43.029；所选稳定帧：00:04:10 与 00:04:34。

Listing 2: 扩展与工具注册的结构性片段

扩展文件可以进入三种加载范围：

表 2: 视频展示的扩展加载范围

范围	路径或入口	适用含义
全局作用域 (global scope)	<code>~/.pi/agent/extensions/*.ts</code>	用户级加载, 可跨项目使用
项目级作用域 (project scope)	<code>.pi/extensions/*.ts</code>	由当前项目约定目录发现
临时加载	<code>pi-e./extensions/change-context.ts</code>	只为本次启动显式加入文件

全局作用域表示资源的发现范围更广, 并不自动赋予更高可信度或更大权限。项目级作用域便于把行为与仓库共同管理, 也仍需代码审查。临时加载适合验证单个文件, 长期使用时还应明确版本和来源。

自动发现目录中的扩展可通过 `/reload` 重载。视频主要采用退出后重新启动 `pi` 的方式, 并通过启动界面的 [Extensions] 列表检查文件是否已被发现。文件已经保存和运行时已经重载属于两个状态; 只检查磁盘文件会遗漏旧进程仍在用旧定义的情况。

案例总览画面与第一章的定位 README 以及本章正文重复, 因此不再占用版面。扩展注册与加载作用域保留为两张证据图: 前者证明工具定义, 后者证明文件如何被运行时发现。

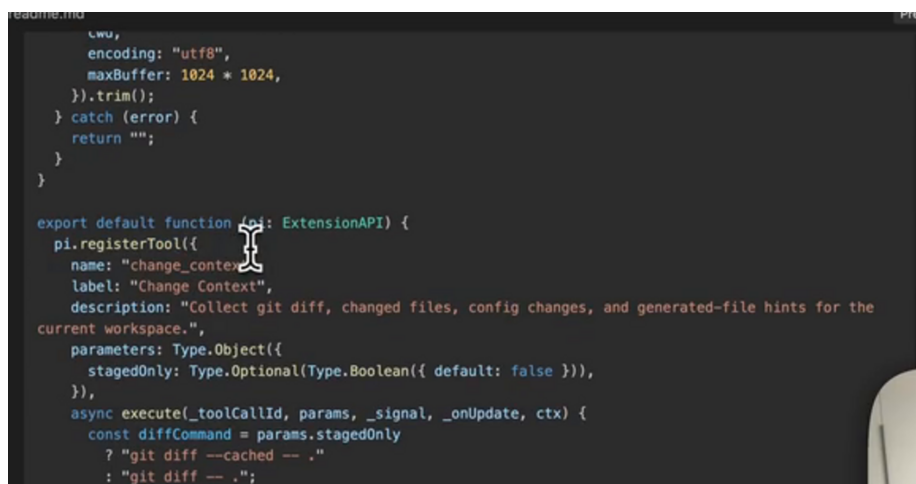


图 5: `change_context` 的注册画面。可辨认内容包括 `pi.registerTool`、工具名、`stagedOnly` 参数, 以及按参数选择暂存或未暂存 diff 的分支。精确可复制命令以正文代码块为准。

5

工具注册回答“运行时获得什么能力”, 加载范围回答“这份能力由哪个环境发现”。两者需要分别核验, 才能避免把代码定义与运行时生效混为一个状态。

⁵ 视频来源时间: 00:07:01.310–00:07:16.229; 所选稳定帧: 00:07:04。



图 6: 扩展加载文档同时展示全局目录、项目目录、-e 临时入口和 /reload。画面把“文件放在哪里”和“修改后怎样生效”放在同一证据链中。

6

2.3 change_context: 从 Git 到模型

change_context 的职责很窄：读取当前项目的代码变更，把变更文件和 diff 返回给模型。它没有直接修改仓库，也没有替模型完成影响判断。这个边界让工具成为后续任务链的可靠数据入口。

视频画面能够确认一个布尔参数 stagedOnly。当该参数为 false 时，工具读取未暂存差异；当它为 true 时，工具读取暂存区差异。相应的只读 Git 命令包括：

```
1 git diff -- .
2 git diff --name-only -- .
3 git diff --cached -- .
4 git diff --cached --name-only -- .
```

Listing 3: 工具使用的只读 Git 上下文采集命令

两组命令分别回答两个问题：--name-only 给出“哪些文件发生变化”，普通 diff 给出“具体改了些什么”。模型获得这两类信息后，可以概括变更、推断影响模块并指出人工复核点。视频用 Package.swift 中的一处简单修改制造工作区差异，使调用结果容易被观察。

工具与模型的职责分界

工具负责采集确定性的 Git 证据；模型负责解释这些证据。模型会根据名称、描述和参数决定是否调用工具以及怎样传参，这种调用模式与视频提到的 MCP 工具相似。工具输出仍需限制范围、处理错误并保护敏感内容，这些生产化细节未在演示中展开。

这个例子有三项教学价值。第一，输入来自真实工作区，结果可以独立核对；第二，stagedOnly 让调用者显式选择变更边界；第三，返回“文件列表 + diff”后，上层技能可以继续规定分析方法，无需把业务判断硬编码进采集工具。

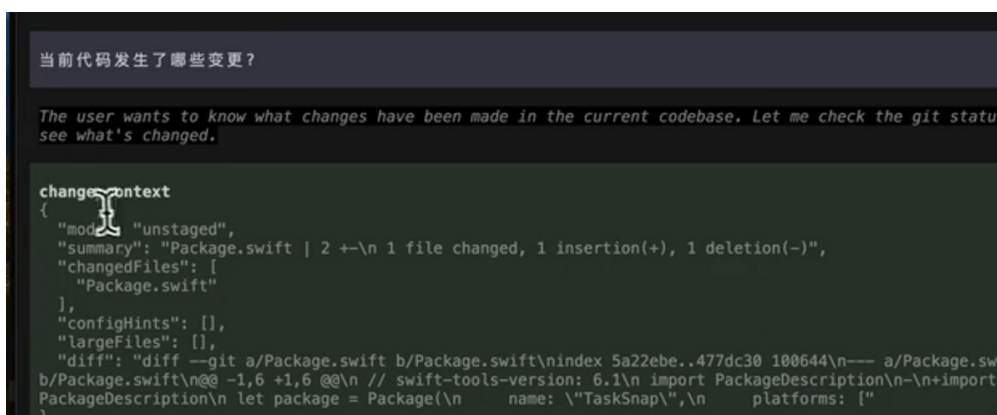
工具读取 diff 也存在隐藏风险：变更中可能包含凭据、个人信息或体积巨大的生成文件。视频没有给出完整的输出预算、脱敏和错误处理方案。生产实现应把这些项目作为扩展边界的一部分进行设计，并保留失败可见性。

2.4 验证闭环：加载提示只是第一项证据

视频的验证顺序可以整理为五个互相独立的检查点：

⁶视频来源时间：00:07:21.570–00:08:22.340；所选稳定帧：00:07:28。

1. **定义**：TypeScript 文件中注册了名称为 `change_context` 的工具；
2. **发现**：文件位于约定目录或通过 `-e` 显式加载；
3. **重载**：启动界面的 [Extensions] 列表出现该扩展；
4. **调用**：用户询问当前代码变更后，模型实际选择了 `change_context`；
5. **结果**：输出包含 `Package.swift` 的变更信息、摘要和 `diff`，且与工作区事实一致。



```
当前代码发生了哪些变更？

The user wants to know what changes have been made in the current codebase. Let me check the git status and see what's changed.

change_context
{
  "mode": "unstaged",
  "summary": "Package.swift | 2 +- \n 1 file changed, 1 insertion(+), 1 deletion(-)",
  "changedFiles": [
    "Package.swift"
  ],
  "configHints": [],
  "largeFiles": [],
  "diff": "diff --git a/Package.swift b/Package.swift\nindex 5a22ebe..477dc30 100644\n--- a/Package.swift\n+++ b/Package.swift\n@@ -1,6 +1,6 @@\n // swift-tools-version: 6.1\n import PackageDescription\n\n let package = Package(\n     name: \"TaskSnap\", \n     platforms: [\"
```

图 7：模型实际调用 `change_context` 后返回结构化结果：模式为未暂存变更，文件列表包含 `Package.swift`，并附带摘要与 `diff`。它把“工具已加载”推进为“工具已运行且结果可核对”。

7

这条证据链可以定位故障所在。扩展未出现在列表中，问题位于发现或加载；扩展已加载但模型未调用，问题可能位于工具描述、请求表达或模型选择；工具已调用但结果为空，问题应回到 Git 状态、参数分支或执行错误。把状态分开后，“看起来已经配置好”就能转化为可诊断的工程流程。

首个工具的五步验收

定义接口，确认发现，完成重载，观察调用，核对结果。五步全部成立，才可以把首个扩展视为真正生效。

2.5 本章小结

本章完成了能力阶梯的第一步：安装和 `/login` 接通模型，TypeScript 扩展把 `change_context` 暴露给运行时，模型调用工具后返回可核对的 Git 变更证据。关键收获是把“文件存在”“扩展加载”“工具调用”“结果正确”拆成独立状态。

下一章将在这个基础上处理更强的运行时控制：会话事件如何提供观察点，危险工具调用如何被确认和阻塞，以及模型供应商的注册、默认选择和实际路由为何需要分别验证。

3 把运行时变成可观察、可拦截、可验证的系统

运行时扩展的工程价值，可以压缩成一句话：把原本藏在智能体内部的关键状态，转化为开发者能够观察、介入和复核的边界。本章继续展开图 2 的运行时层，并建立两条闭环：危险动作必须

⁷ 视频来源时间：00:08:38.330-00:09:29.990；所选稳定帧：00:09:18。

经过“策略前置、人工确认、明确阻塞、结果复核”，模型路由必须经过“注册、认证、默认选择、实际请求、供应商日志”。学完后，读者应能区分“配置看起来完成”与“目标行为已经发生”。

本章核心判断

生命周期事件负责暴露运行节点，工具调用拦截负责在执行前施加策略，斜杠命令负责提供确定且可观察的触发入口，供应商日志负责给出独立于终端回答的路由证据。任何一个环节成功，都不能替代其余环节的验证。

3.1 生命周期事件是运行时挂点

生命周期事件（lifecycle event）把连续运行过程切成可识别的节点。扩展可以在这些节点记录事实、收集指标，或者在满足条件时介入控制。视频用两个事件说明这种运行时挂点：

```
1 pi.on("session_start", async (_event, _ctx) => {
2   // session-start logging or metrics
3 });
4
5 pi.on("tool_call", async (event, ctx) => {
6   // observe the call and apply a policy when needed
7 });
```

Listing 4: 画面可确认的生命周期事件结构

`session_start` 适合记录会话启动、累计会话次数或发出指标。它回答的是“何时进入一次新的运行过程”。`tool_call` 面向具体工具调用，回答的是“模型准备执行什么动作，以及此刻是否需要介入”。两者分别覆盖会话级和动作级边界，同属于观察面，粒度并不相同。



图 8: 同一扩展中的三个运行边界：`session_start` 记录会话启动，`tool_call` 在命令执行前请求确认，`registerCommand` 暴露确定的斜杠入口。⁸

日志本身只证明处理器被触发。若扩展的目标是阻止危险动作，还需要从被动监听继续走到执行控制，并验证目标动作确实没有发生。

⁸ 视频来源时间区间：00:09:43.540–00:10:27.050。画面从该区间七个密集候选帧中选择 00:10:23 的完整代码状态，并裁去桌面、字幕和人物区域。

3.2 危险动作确认闭环

工具调用拦截 (tool-call interception) 指在工具执行前检查调用, 并允许确认、修改或阻塞。视频中的教学示例检查 `bash` 工具的命令字符串; 当字符串含有 `rm` 时, 界面调用 `ctx.ui.confirm(...)` 请求确认。用户选择拒绝后, 处理器返回带有阻塞语义的结果, 随后再检查测试文件仍然存在。

这个案例的重点不在删除命令本身, 而在四个可验证步骤:

1. **策略前置**: 在工具真正执行前识别潜在危险动作。
2. **人机确认**: 把最终决定暴露给用户, 界面明确展示风险。
3. **明确阻塞**: 拒绝后返回阻塞结果, 使模型知道调用未被执行。
4. **结果复核**: 通过文件仍然存在这一外部事实, 确认策略产生了预期效果。



图 9: 危险动作闭环的两端证据: 左图显示执行前的 Yes/No 确认; 右图显示用户拒绝后返回阻塞语义, 随后通过只读列表确认目标文件仍存在, 并显示 `/hello` 的 `Hello world!` 输出。⁹

字符串规则只适合解释机制

把“命令字符串包含 `rm`”当作危险判据, 会同时产生误报和漏报。别名、脚本封装、间接调用和参数组合都可能绕过它; 普通文本中偶然出现相同字符也可能触发确认。生产策略还需要结构化命令解析、动作分类、允许路径边界、权限控制、审计记录和事后验证。

因此, 安全扩展的验收对象应当是“动作是否受控”这一外部结果。只看到确认框, 无法证明阻塞路径正确; 只看到测试文件存在, 也无法证明所有同类危险动作都会被拦截。视频证明了最小闭环, 生产系统仍需扩大策略覆盖并测试绕过路径。

3.3 斜杠命令是可验证的运行入口

视频通过 `pi.registerCommand("hello", ...)` 注册 `/hello`。这个示例很小, 却揭示了智能体运行框架中一个重要边界: 自然语言请求由模型解释, 斜杠命令则把某个操作暴露为用户能够明确选择的入口。

斜杠命令与可观察、可验证的运行边界存在直接关系:

- **入口可观察**: 命令出现在交互界面的命令列表中, 证明注册结果已经被运行时发现。
- **触发可定位**: 用户输入 `/hello`, 可以把后续行为归因到对应命令处理器。

⁹视频来源时间区间: 00:11:39.370–00:12:53.910。左、右画面分别取自 00:12:14 与 00:12:50; 两个子区间共检查十二个密集候选帧。

- **结果可验证：**界面输出 Hello world!，证明处理器被执行并返回了可见结果。
- **边界仍有限：**一次正确输出只证明这个最小处理器可用，不能证明权限、错误处理、重试或业务语义已经达到生产要求。

为什么要保留这个最小示例

/hello 的教学价值是证明“注册—发现—触发—结果”边界。后续的 /assess-change 会复用同一类可发现入口，同时增加技能、工具与输出合同。这样，斜杠命令从机制证明逐步成为可复用任务入口。

扩展文件改变后，视频通过退出并重启 pi agent 完成重新加载；本地文档同时说明自动发现目录可使用 /reload。因此，命令未出现时应先检查扩展是否被重新加载，再检查注册逻辑。把“源文件已经保存”直接等同于“命令已经生效”，会跳过运行时发现这一状态。

3.4 自定义模型供应商先完成注册

模型供应商负责描述请求要发往何处、如何认证、使用何种 API 适配，以及有哪些模型能力。视频使用本地 LiteLLM 作为模型网关，并以 pi.registerProvider("litellm", {...}) 展示注册结构。画面可确认的字段可以按职责分为五组：

职责	字段与含义
供应商标识	注册 ID 与展示名称，使 LiteLLM 出现在供应商选择界面
连接位置	baseUrl 指向兼容接口网关
认证契约	apiKey 使用环境变量式契约或登录输入；正文不保存凭据值
协议适配	api 声明 OpenAI-compatible 请求的适配形式
模型元数据	模型 ID、展示名、推理支持、输入类型、成本、上下文窗口和最大输出

演示中模型只声明文本输入，成本字段全部写为零。前者是该示例模型的能力声明，后者只是演示简化，不能据此推断真实计费。视频口述模型名为 DeepSeek V4 Flash，但 480p 画面中的精确 defaultModel 字符串局部被光标遮挡，正文不补写未经确认的完整代码标识。

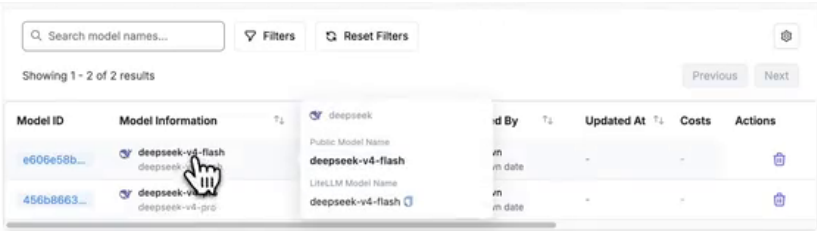


图 10: LiteLLM 模型管理页显示两个 DeepSeek 模型条目。该画面只证明网关侧存在可注册模型，不能证明 pi agent 的默认模型已经切换。¹⁰

随后出现的扩展代码把连接、认证、协议与模型能力组织进同一注册对象。画面中的环境变量式认证契约没有暴露真实凭据值。

¹⁰视频来源时间区间：00:13:41.439–00:13:58.330。画面从七个密集候选帧中选择 00:13:50，并只保留模型列表区域。

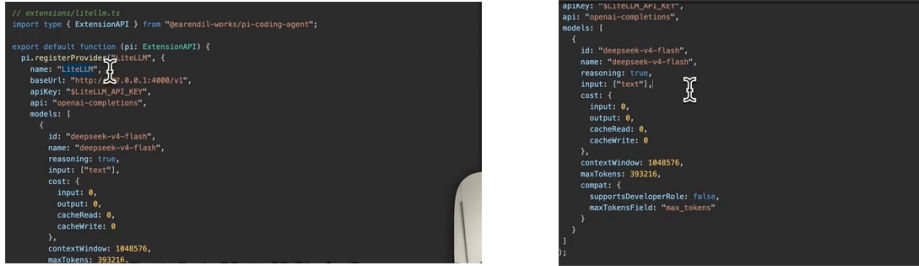


图 11: `registerProvider` 配置的上下两段：上段展示供应商标识、网关地址、认证契约与协议适配；下段展示输入类型、成本、上下文窗口、最大输出和兼容字段。图注不复写被光标遮挡或分辨率不足的精确值。¹¹

扩展重载后，[Extensions] 出现 `litellm.ts`，`/login` 的供应商选择器出现 LiteLLM。这些证据足以证明“供应商已注册且可配置”，仍然不足以证明下一次模型请求会经过 LiteLLM。

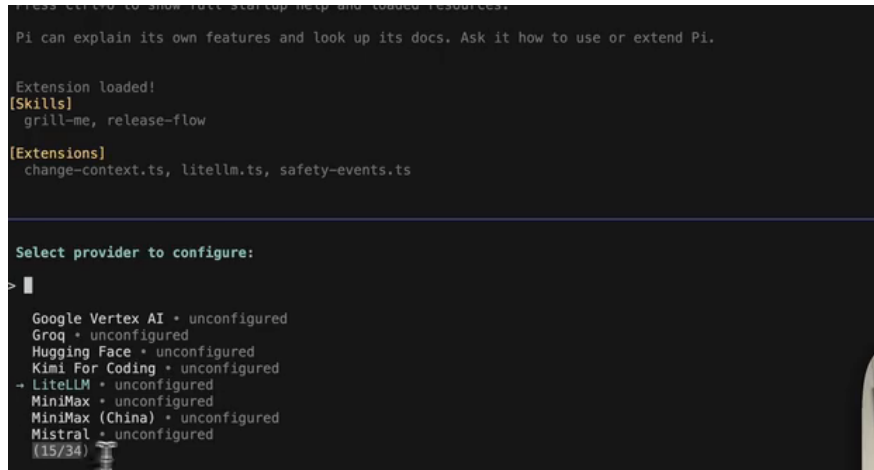


图 12: 注册证据的完整状态：启动资源列表已加载 `litellm.ts`，`/login` 的供应商选择器同时出现 LiteLLM；凭据输入步骤未纳入画面。¹²

3.5 四个状态必须分别验证

自定义供应商演示中最有价值的部分，是一次“终端能回答、目标网关却没有新日志”的失败诊断。它说明模型供应商的注册、认证、默认供应商（default provider）选择和实际路由是四个独立状态：

状态	它说明什么	视频中的可验证证据
注册	运行时认识该供应商及其模型配置	LiteLLM 出现在 <code>/login</code> 选择器
认证	配置过程已保存所需认证信息	<code>/login</code> 流程能够完成
默认选择	后续请求默认指向哪个供应商和模型	<code>settings.json</code> 中的 <code>defaultProvider</code> 、 <code>defaultModel</code>
实际路由	某次具体请求真正到达目标网关	LiteLLM 最新请求日志含对应提示与响应

¹¹ 视频来源时间区间：00:14:01.180–00:15:15.260。两个代码子区间共检查十个候选帧，最终选择 00:14:24 与 00:14:54 的互补完整状态。

¹² 视频来源时间区间：00:15:32.870–00:16:07.070。共检查六个候选帧，最终选择 00:15:57；00:16:02 的凭据输入页被明确排除。

视频第一次完成 LiteLLM 登录后，终端仍能正常回答，LiteLLM 日志却没有新增记录。原因是 `~/.pi/agent/settings.json` 仍保留此前的 DeepSeek 默认设置。修改默认供应商与默认模型、重启 pi agent、连续发送探针后，LiteLLM 日志出现新请求；打开最新记录，能够看到探针与对应响应，路由闭环才算完成。



图 13: settings.json 的持久化状态变化：左图的 defaultProvider 仍指向原供应商，右图已改为 LiteLLM。defaultModel 行虽然可见，480p 画面不足以支持可靠的逐字符复写，正文保留这一未决项。¹³

持久化设置只完成“默认选择”这一层。下一步仍需从目标网关取得与探针相互对应的请求证据。

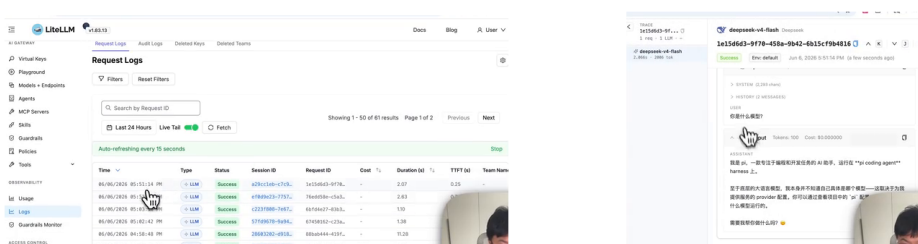


图 14: 供应商侧路由证据：左图是刷新后的 LiteLLM 请求日志列表，右图打开最新记录并显示用户探针与响应正文。画面只承担请求存在性和对应关系证明，不复写请求 ID 或其他运行元数据。¹⁴

自定义供应商：状态推进与证据落点

教学重构 | 依据视频 00:15:48–00:17:45

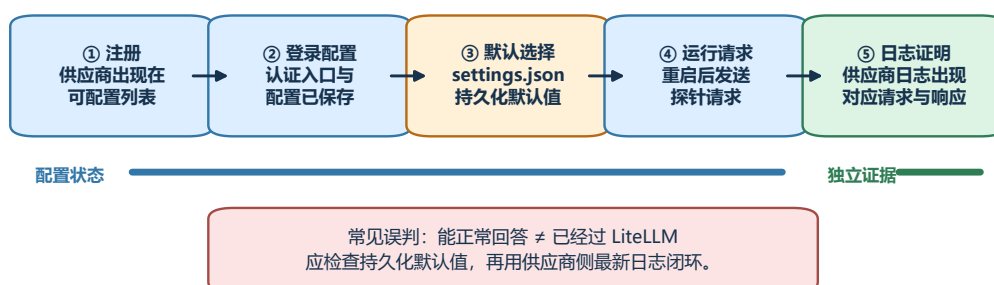


图 15: 自定义供应商的状态推进与证据落点（教学重构，依据视频 00:15:48–00:17:45）。前四步形成配置状态，第五步才提供目标系统边界上的独立证据。

¹³ 视频来源时间区间：00:16:44.000–00:17:06.040。共检查六个候选帧，选择 00:16:47 与 00:16:57 的修改前后状态。

¹⁴ 视频来源时间区间：00:17:14.920–00:17:45.380。共检查六个候选帧，最终选择 00:17:33 的日志列表和 00:17:39 的请求详情。

路由验证原则

终端出现正常回答，只证明某个模型路径可用。目标供应商的最新请求日志与探针相互对应，才证明这次请求经过了预期路由。验证必须落在目标系统的边界上。

3.6 可视化界面必须暴露持久化状态

如果团队基于 pi agent 构建可视化界面，供应商选择器至少要区分“可用供应商”和“默认供应商”。只允许用户保存凭据，却隐藏 `defaultProvider` 与 `defaultModel`，会制造一种危险错觉：界面显示 LiteLLM 已配置，实际请求仍沿旧默认路由执行。

一个可验证的界面至少需要呈现三类信息：

- 当前默认供应商和默认模型；
- 保存或切换后的持久化结果，以及重新加载要求；
- 一次探针请求对应的供应商侧证据或明确的诊断入口。

这里的设计推论来自视频暴露的状态边界。视频没有提供完整 GUI、权限模型或审计实现，因此正文只给出状态暴露与验证原则。

3.7 本章小结

可控运行时依靠两条闭环。危险动作闭环是“策略前置 + 人机确认 + 明确阻塞 + 结果复核”；模型路由闭环是“注册 + 认证 + 默认选择 + 实际请求 + 供应商日志”。斜杠命令位于这套体系的可验证入口处：它让处理器可发现、触发可定位、结果可观察。下一章将在这个入口之上加入任务方法与输出合同，使一次成功演示成为能够反复调用的任务。

4 让一次演示变成可复用任务：技能与提示模板

一次自然语言请求可以成功，并不代表团队已经拥有可复用任务。稳定复用需要三层职责同时清晰：工具提供事实数据，技能规定分析方法，提示模板固定触发入口、调用顺序与输出合同。本章沿图 2 的任务方法层与调用编排层展开，并复用图 7 已核验的数据入口，说明 `change_context` → `change-assessment` → `/assess-change` 如何把临时请求升级为可发现、可复用、可核验的任务入口。

三层职责

工具回答“取得什么数据”；技能回答“按什么方法工作”；提示模板回答“如何稳定触发，以及交付什么格式”。三者可以组合，职责不能互相替代。

4.1 渐进加载控制任务说明的读取时机

渐进加载（progressive loading）把技能的发现与完整读取分成两个阶段：运行时先扫描名称和描述，等用户请求与某项技能匹配后，再读取该目录中的完整 `SKILL.md`。这种机制使许多任务说明可以同时存在，同时避免一开始就把每份完整指令全部装入上下文。

视频能够证明的范围只有这两个阶段：

1. 扫描技能的 `name` 与 `description`，用于发现和匹配；
2. 请求匹配后读取完整 `SKILL.md`，按其中的方法与输出要求执行。

每项技能使用独立目录，目录内必须有 `SKILL.md`。名称帮助系统和用户识别任务，描述决定何时应使用它，正文则承载完整方法。视频没有说明缓存、冲突优先级或多个同名技能的覆盖算法，这些行为不能从“渐进加载”一词推导出来。



图 16: 本地文档的完整技能说明状态：渐进加载的“先扫描名称与描述、匹配后读取完整内容”与项目、全局、pi 扩展包、临时四种来源同时可见。¹⁵

4.2 四种来源决定复用范围

技能的内容可以相同，加载来源决定它面向当前项目、多个项目、一次临时运行，还是随 pi 扩展包分发。

来源	视频确认的位置或入口	适合的使用边界
项目	<code>.pi/skills/</code>	与当前仓库规则和代码结构绑定的任务
全局	<code>~/.pi/agent/skills/</code> 、 <code>~/.agents/skills/</code>	跨项目反复使用的个人或共享任务方法
临时	<code>pi --skill <path></code>	单次测试、未进入稳定目录的技能
pi 扩展包	包内 <code>skills/</code>	随版本化分发单元交付给团队

项目级作用域与全局作用域分别回答“只在此项目使用”与“跨项目可用”。全局并不意味着更可信或权限更高；它只描述发现范围。pi 扩展包来源则把加载问题与团队分发连接起来，第 5 章将继续讨论包清单与安装后验证。

Hermes 对照的正确用法

视频展示 `hermes skills -h`，其中含搜索、安装、检查、列表、更新和审计等管理入口。这个画面用于说明技能管理仍有可扩展空间。它不能作为 pi agent 已经拥有同一组命令的证据；若要在 pi agent 中获得类似入口，需要另行注册命令并实现对应行为。

¹⁵ 视频来源时间区间：00:18:13.830–00:18:44.990。共检查七个密集候选帧，选择 00:18:44 并裁去桌面、字幕与人物区域。

4.3 change-assessment 固化评估方法

change-assessment 把“评估当前工作区变更”从一句临时要求，固化为带有明确目标和稳定输出的技能。视频中的任务合同包含四个结果组：

1. **改了什么**：概括文件和行为变化；
2. **影响范围**：指出可能受影响的模块、调用方或使用场景；
3. **风险点**：识别容易出错、需要警惕或证据不足的位置；
4. **复核建议**：给出人工检查与验证方向。

一个最小 SKILL.md 先用 frontmatter 声明名称和用途，再在正文中定义评估步骤与四段输出。下面只呈现视频画面与分析报告能够确认的合同层结构：

```
1 ---
2 name: change-assessment
3 description: Assess workspace changes with impact, risks,
4   and review notes.
5 ---
6
7 Return:
8 - What changed
9 - Impact scope
10 - Risk points
11 - Review recommendations
```

Listing 5: change-assessment 的最小任务合同

视频在全局技能目录中创建独立的 change-assessment 目录和 SKILL.md，重启 pi agent 后，[Skills] 列表出现该技能。它既可以被匹配的自然语言请求触发，也可以通过 /skill: change-assessment 直接调用。输出按四组合同组织，证明技能已被发现、被执行，并约束了结果结构。



图 17: change-assessment 的资产形态：独立目录、必需的 SKILL.md、frontmatter 名称与描述均 在同一完整画面中出现。¹⁶

资产定义完成后，还要分别证明运行时发现了技能，并且执行结果遵守四段合同。



图 18: 技能的发现与执行证据：左图的 [Skills] 列表出现 `change-assessment`，右图结果按“改了什么、影响范围、风险点、建议复核”四组组织。¹⁷

需要注意，技能规定的是工作方法，并不天然拥有工作区变更数据。若它只依赖模型自行猜测上下文，四段输出格式仍然可以整齐，事实基础却可能不足。`change_context` 工具正是为这一缺口提供结构化 Git 变更。

4.4 提示模板把方法封装成稳定入口

提示模板位于调用编排层。它可以细化一项技能的交付要求，也可以组合多项技能与工具，并以文件名形成可发现的斜杠命令。项目和全局目录分别是：

- 项目：`.pi/prompts/`
- 全局：`~/.pi/agent/prompts/` 或 `~/.agents/prompts/`

视频创建 `assess-change.md`，加载后在 [Prompts] 中显示 `/assess-change`。它的核心合同是先使用 `/skill:change-assessment`，在工具可用时先调用 `change_context` 获取结构化上下文，最终返回四段评估结果。

```
1 Use /skill:change-assessment to assess the current
2 workspace changes.
3
4 If tools are available, call change_context first to
5 obtain structured context.
6
7 Return:
8 - What changed
9 - Impact scope
10 - Risk points
11 - Review recommendations
```

Listing 6: `/assess-change` 的可确认编排合同

`description` 是入口契约的一部分。命令选择器中的用户尚未读到模板正文，需要依靠描述判断这个命令解决什么问题。文件名负责形成 `/assess-change`，描述负责解释用途，正文负责固定调用与交付要求。

¹⁶视频来源时间区间：00:20:04.159–00:20:41.879。共检查七个候选帧，选择 00:20:29；00:20:35 的选中文本状态因遮挡和视觉噪声被排除。

¹⁷视频来源时间区间：00:20:42.920–00:21:12.050。共检查七个候选帧，选择 00:20:49 的加载列表和 00:21:09 的完整四段结果。

```
====
description: 评估当前工作区的代码变更
====

请使用 /skill:change-assessment 评估当前工作区的代码变更。

如果可以使用工具，请先调用 change_context 获取结构化上下文。
输出格式：

## 改了什么
## 影响范围
## 风险点
## 建议复核
~
~
~
~
```

图 19: assess-change.md 的完整编排合同：description、技能调用、可用时先调用 change_context，以及四段输出标题均清晰可见。¹⁸

模板保存并重启后，发现证据和工具调用证据分别落在命令列表与运行记录中。

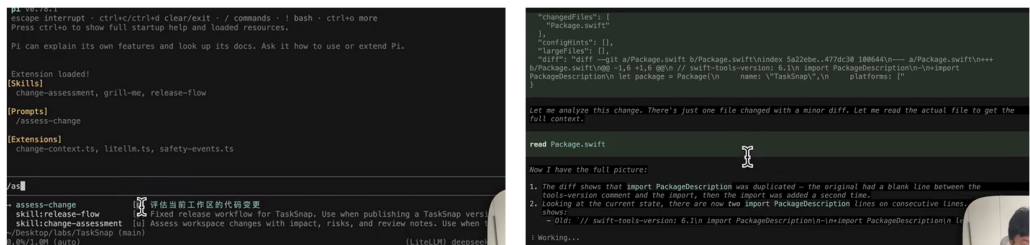


图 20: 提示命令的发现与执行证据：左图显示 [Prompts] 中的 /assess-change 及命令选择器描述，右图显示执行中取得 Package.swift 的结构化变更并继续读取文件。¹⁹

这里再次出现第 3 章的“入口—观察—验证”关系：

- /assess-change 出现在 [Prompts] 中，证明模板已被发现；
- 用户明确调用命令，触发来源可定位；
- 运行过程调用 change_context，数据来源可观察；
- 结果具有四个固定部分，交付合同可验证。

正常输出仍然不能独自证明工具确实提供了当前 Git 数据。可靠复核应检查工具调用记录与结果中的文件、diff 摘要是否对应当前工作区。稳定入口降低了调用歧义，证据仍需落在每一层自己的边界上。

4.5 三层案例：数据、方法与入口各司其职

把本章案例连起来，可以看到同一个任务由三层互补能力完成：

如果只写工具，用户每次仍要重新说明分析方法和输出格式；如果只写技能，它可能缺少可靠的当前变更数据；如果只写提示模板，模板会变成把临时提示换了一个文件名。三层组合的价值在于把事实获取、推理方法和交付入口分别固化，并允许每一层单独验证与替换。

¹⁸ 视频来源时间区间：00:22:08.590–00:22:48.400。共检查七个候选帧，选择 00:22:39 的完整文件状态。

¹⁹ 视频来源时间区间：00:22:51.160–00:23:33.310。共检查七个候选帧，选择 00:23:10 的命令发现状态和 00:23:22 的工具结果状态。

层级	案例资产	不可替代的职责
运行时数据层	change_context 工具	读取 Git 变更文件与 diff，提供结构化事实
任务方法层	change-assessment 技能	规定如何分析影响、风险和复核点
调用编排层	/assess-change 提示命令	固定工具与技能的调用关系，并约束四段输出

工具、Skill 与提示命令：职责和数据流

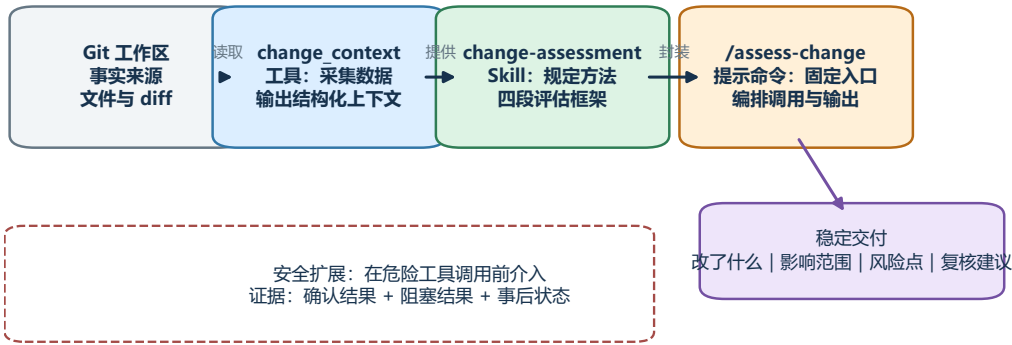


图 21: 工具、技能与提示命令的职责和数据流（教学重构，依据视频 00:07:01–00:23:33）。每一层都拥有独立输入、输出与完成证据；安全扩展位于危险工具调用前。

三个常见混淆

TypeScript 扩展承担运行时 API 与事件行为，SKILL.md 承担任务方法，提示模板承担调用编排。目录相似或都能影响模型输出，不表示三者可以互换。判断资产类型时，应先问它拥有的是运行时行为、任务方法，还是入口与输出合同。

4.6 何时写扩展、何时写技能、何时写提示模板

面对一个新需求，可以按所拥有的责任边界选择实现位置：

需求信号	合适资产	完成证据
需要读取外部状态、注册工具、监听事件或控制执行	扩展与工具	运行时发现、真实调用、结构化结果或阻塞结果
需要固定判断方法、步骤、质量要求和任务知识	技能	技能被发现，输出遵守方法与合同
需要一个易发现的命令、组合多个能力、稳定输出格式	提示模板	命令可见、调用顺序可观察、格式可核验

这张职责表也给出了调试顺序。数据不对，先查工具边界；分析方法不稳定，查技能；命令找不到或输出格式漂移，查提示模板的目录、描述与正文。分层以后，问题可以定位到拥有该职责的资产。

4.7 本章小结

可复用任务由三层合同共同构成：工具提供当前且结构化的数据，技能固化分析方法，提示模板提供可发现的斜杠入口并约束调用与输出。`change_context` → `change-assessment` → `/assess-change` 证明这三层既能组合，也能分别观察和验证。下一章将把这些个人目录中的资产连同主题与扩展一起装入 pi 扩展包，使任务从个人复用进入团队分发。

5 从个人配置到团队交付：主题与 pi 扩展包

主题与 pi 扩展包解决的是两类不同的一致性问题的：主题统一终端中的颜色表达，pi 扩展包统一团队实际安装和加载的能力资产。前者的验收对象是一份完整、可切换的颜色配置；后者的验收对象是一组目录、一个明确清单，以及安装后的加载证据。完成这两层之后，前几章建立的工具、技能与提示模板才从个人机器上的配置，变成能够交给团队复用的交付物，并补全图 2 的团队分发层。

团队交付的完成条件可以压缩为四步：目录完整、清单明确、安装成功、资源加载。只看到文件已经复制，或者安装命令返回成功，都不足以单独证明团队成员已经获得预期能力。

5.1 主题的能力边界：统一颜色表达

主题是一份完整的终端颜色契约。视频展示的配置覆盖输入、输出、思考文本、分隔线、边框等颜色 token；切换主题后，这些颜色会立即变化。它适合解决“同一类信息应以什么颜色出现”这一视觉一致性问题。视频没有展示布局重排、交互流程改造或新控件，因此这些能力不能归入主题。

可以把主题理解为编辑器的配色方案：它改变语法元素和界面边界的视觉编码，却不改变编辑器提供哪些命令，也不改变一次任务怎样执行。这个边界很重要，因为“界面颜色一致”和“团队 workflow 一致”拥有不同的验收对象。

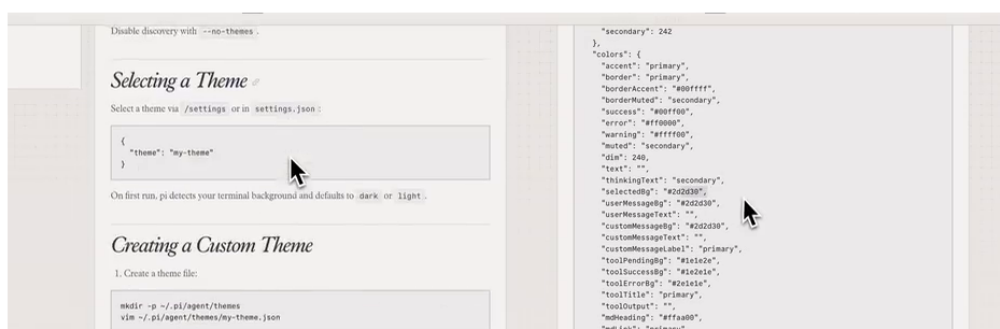


图 22: 官方主题文档把主题选择、全局主题目录和完整颜色字段放在同一配置合同中：左侧可见 `~/ .pi/agent/themes/`，右侧显示颜色 token 集合只是完整 JSON 的一部分。²⁰

5.1.1 完整 JSON 是配置合同

视频中的官方主题文档给出全量 JSON 模板，并强调应复制完整模板，再修改需要调整的字段。只留下少量已修改字段会形成残缺配置，无法满足主题加载所需的完整 token 集合。安全的理解方式如下：

²⁰ 视频来源时间区间：00:24:02.720-00:24:36.200。

1. 从完整模板开始，保留所有必需字段；
2. 只修改有明确视觉意图的颜色值；
3. 将主题文件放入 `~/pi/agent/themes/`；
4. 重启 pi agent，使新增主题进入可选列表；
5. 在设置中的主题选择器里切换，并观察输入、输出、分隔线与边框是否按预期变化。

主题文件的危险点是“局部看起来合理，整体合同已经破损”。480p 画面无法支持逐字抄录所有小字号 token 值，因此本文只保留完整性原则和已确认的目录，不补写无法辨认的字段值。

视频分别展示了 `dark`、`light` 和 `custom-theme` 的选择状态。切换后颜色效果可以立即观察，因此这里形成了一个很短的验证闭环：文件被发现、主题可选择、颜色发生预期变化。字幕把打开选择器的入口识别成了一个具体斜杠命令，画面没有留下足够证据确认该命令；实际使用时应以当前版本的设置入口为准。

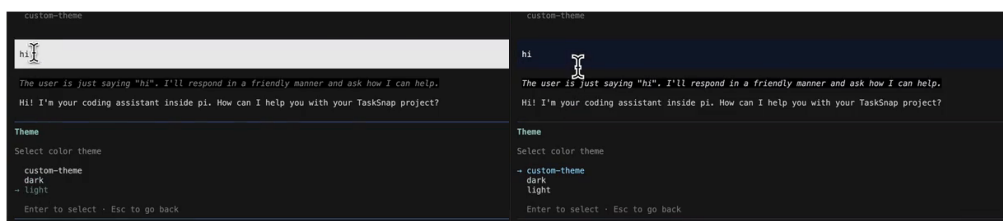


图 23： 同一终端内容下的主题切换对比：左侧选择 `light`，输入区域变为浅色；右侧选择 `custom-theme`，输入与选中项采用自定义深蓝色。画面只证明颜色即时变化，不支持交互结构重设计。²¹

5.2 pi 扩展包：把能力资产变成分发单元

pi 扩展包是一种由目录和 `package.json` 清单共同定义的安装单元。它可以组织 `extensions/`、`skills/`、`prompts/` 和 `themes/`，也可以只包含其中一类或几类资源。它的价值来自一项明确声明：“这个交付物包含什么、安装后应加载什么”。

前文的代码变更评估案例可以由此完成团队级闭环：

- `change_context` 工具采集 Git 变更；
- `change-assessment` 技能固定评估方法；
- `/assess-change` 提示命令固定调用顺序和输出结构；
- 安全扩展约束危险动作；
- 主题统一终端颜色；
- pi 扩展包把这些资源交给团队安装。

²¹ 视频来源时间区间：00:25:24.170–00:26:14.450。

这几层承担互补职责。扩展改变运行时行为，技能提供任务方法，提示模板提供稳定入口和输出合同，主题提供颜色约定，pi 扩展包负责组织与分发。把它们混成一个“配置文件”概念，会让维护者无法判断故障究竟发生在运行时、任务编排、入口发现、视觉呈现还是安装分发。

5.2.1 目录表达内容，清单表达意图

视频先建立包目录，再把需要分发的资源放入相应子目录，最后通过 `package.json` 顶层的 `pi` 对象声明资源位置。下面的结构只复现画面和分析报告能够确认的字段关系：

```
1 {  
2   "name": "my-pi-package",  
3   "keywords": ["pi-package"],  
4   "pi": {  
5     "extensions": ["/extensions"],  
6     "skills": ["/skills"],  
7     "prompts": ["/prompts"],  
8     "themes": ["/themes"]  
9   }  
10 }
```

Listing 7: pi 扩展包的清单结构示意图

清单应忠实描述实际内容。某类资源不存在时，应省略对应字段；无需为了追求表面完整而声明空目录。这个原则使目录和 manifest 互相校验：目录回答“文件实际在哪里”，pi 对象回答“安装器应该把哪些目录当作 pi agent 资源”。



图 24: 演示包的实际 `package.json`：顶层 `pi` 对象只声明当前包中存在的 `skills`、`prompts` 和 `themes` 目录，直接体现“内容存在才进入清单”的原则。²²

视频说明 pi 扩展包可以来自 npm、Git、HTTP、绝对路径或相对路径。来源解决“怎样取得包”，清单解决“包里哪些目录属于 pi agent 资源”。两者需要分别核验。

5.3 安装之后还要证明资源已加载

本地演示使用 `pi install pi-package` 安装示例包，再使用 `pi list` 查看已安装包及其来源位置。随后重启 pi agent，包内技能出现在加载列表中，并带有演示界面所示的全局标记。这一连串证据比单看“安装成功”更强，因为它覆盖了安装记录、来源位置和运行时资源发现三个状态。

²²视频来源时间区间：00:28:10.780–00:28:50.210。

```
~/Desktop/temp/pi-package ls
package.json skills
~/Desktop/temp/pi-package cd ..
~/Desktop/temp pi install pi-package
Installing pi-package...
Installed pi-package
~/Desktop/temp pi list
User packages:
  ../../Desktop/temp/pi-package
  /Users/morrimo/Desktop/temp/pi-package
~/Desktop/temp

pi v0.78.1
escape interrupt · ctrl+c/ctrl+d clear/exit · / commands · ! bash · ctrl+o more
Press ctrl+o to show full startup help and loaded resources.

Pi can explain its own features and look up its docs. Ask it how to use or extend Pi.

Extension loaded!
[Skills]
change-assessment, git-auto-commit-and-push, grill-me, release-flow

[Prompts]
/assess-change

[Extensions]
change-context.ts, litellm.ts, safety-events.ts

[Themes]
custom-theme

/ski

skill:grill-me [p] Interview the user relentlessly about a plan or design until reach
skill:release-flow [p] Fixed release workflow for TaskSnap. Use when publishing a TaskSn
skill:change-assessment [u] Assess workspace changes with impact, risks, and review notes. Use
skill:git-auto-commit-and-push [u] 基于当前代码修改自动总结 commit message, 执行 git add/commit/push
```

图 25: 安装与运行时发现的连续证据。上半部分显示 `pi install pi-package` 成功且 `pi list` 返回包路径；下半部分显示重启后的资源清单，并在技能选择器中以全局标记 `[u]` 列出包内技能。²³

安装验收可以按下面四个问题逐层推进：

1. **目录完整吗？** 实际资源是否都进入正确的 `extensions/`、`skills/`、`prompts/` 或 `themes/` 目录。
2. **manifest 明确吗？** `package.json` 是否只声明真实存在并准备分发的资源类型。
3. **安装记录存在吗？** 安装过程是否成功，`pi list` 是否显示预期包和来源位置。
4. **资源真的加载了吗？** 重启后，预期的技能、提示模板、扩展或主题是否出现在运行环境中，并能完成各自的最小验证。

团队一致性来自“同一包版本 + 同一清单 + 同一安装后证据”。包名相同或目录内容相似无法单独保证每台机器拥有相同运行行为。

5.4 从演示包走向团队治理

视频证明了本地包可以被安装、列出，并让包内技能进入全局可用状态。团队采用这一机制时，还需要补充视频没有展开的治理约束。这些内容属于工程延伸建议：

- **版本：**为包和关键资源建立可追踪版本，记录变更影响与兼容边界。
- **依赖：**明确运行时依赖、外部命令和最低环境要求，避免“在作者机器上可用”成为隐含前提。
- **权限：**对会执行命令、访问文件或调用外部服务的扩展进行权限审查。

²³ 视频来源时间区间：安装与列表为 00:29:10.490–00:29:31.510；重启与资源发现为 00:29:37.900–00:30:11.650。

- **升级：**在推广新版本前验证加载、核心任务和安全策略；保留可恢复的旧版本。
- **审计：**保存包来源、版本、清单和安装后验证结果，使团队能够解释某次运行使用了哪组资产。

视频演示在替换示例技能时出现了永久删除语法。该动作只属于演示过程，本文不把它作为操作步骤。需要替换资源时，应先将旧资源移动到可恢复的备份或 待删除/ 目录，再安装和验证新版本；验证失败时仍有恢复路径。



图 26: 资源类型与加载作用域的统一视图（教学重构）。矩阵只填写视频在 00:18:13–00:30:11 内能够确认的目录和临时入口；“未展示”明确保留当前证据空白。

5.5 本章小结

主题负责终端颜色的一致表达，验收重点是完整 JSON、成功发现和即时切换；pi 扩展包负责能力资产的一致分发，验收重点是目录、manifest、安装记录和运行时加载。至此，个人机器上的工具、技能、提示模板和安全策略具备了团队交付路径。下一章将把这些离散机制压缩成一条实施路线，并明确“功能出现”之后仍需补齐的独立验证与生产治理。

6 总结与延伸：一条可落地的定制路线

pi agent 定制的完成标准包括五项：选择理由明确、各层职责清晰、危险动作受控、模型路由有独立证据、资产能够稳定分发。功能能运行只是中间里程碑；真正可交付的智能体运行框架，还要让关键状态可观察、结果可复核、团队资产可追踪。

本章将全片压缩成一条六步实施路线，并回引图 15 的路由证据链与图 26 的团队资源边界，说明同一个工程原则：系统的关键结果需要在相应责任边界上留下证据。

6.1 讲者的最终判断：先确定是否需要定制

讲者在 00:30:18–00:31:46 的实质性总结首先回到产品选择：pi agent 是轻量命令行智能体运行框架（Harness Agent）底座，适合准备进行定制开发的使用者。若目标是安装后立即获得一套较完

整的现成功能，可以评估 OpenClaw 或 Hermes。视频只把两者作为预装型方案的对照，没有逐项证明它们的全部能力。

当团队选择 pi agent 时，可用的能力面形成了清晰层级：

- TypeScript 扩展负责自定义工具、会话事件监听、工具调用监听、命令注册和模型供应商注册；
- 技能通过项目级或全局目录固化任务方法；
- 提示模板位于技能之上，封装常用调用、工具协同和输出格式；
- 主题在视频范围内负责终端颜色；
- pi 扩展包借助 package.json 组织这些资源并提供分发入口。

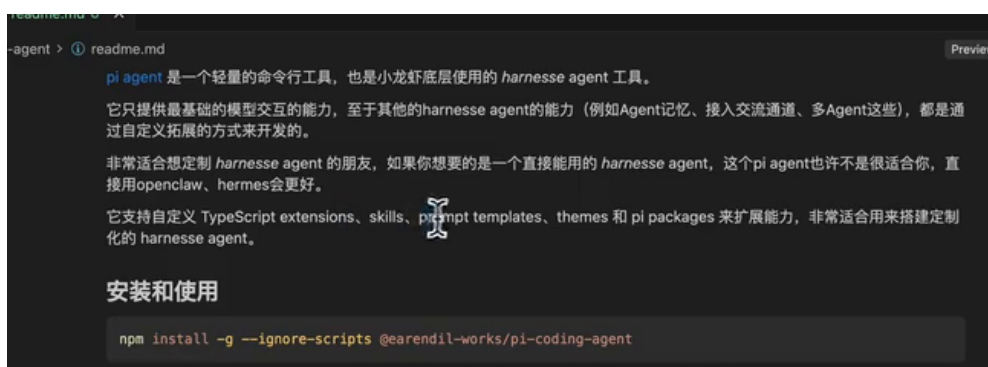


图 27: 讲者结尾 README 的完整总结状态：轻量底座定位、定制适用条件、预装型方案对照，以及 extensions、Skills、prompt templates、themes、pi packages 五类扩展面同时可见。²⁴

一句话选择规则：需要掌控工具、生命周期、安全、模型路由和团队资产时，pi agent 提供了可组合的定制底座；主要目标是立即使用完整现成功能时，应先评估预装型方案。

6.2 六步实施路线：从边界选择到团队核验

6.2.1 第一步：明确要控制的边界

先把需求写成可检查的问题：团队是否需要自定义工具？是否需要在会话或工具调用节点介入？是否需要接入自定义模型供应商？是否要统一技能、提示模板、主题和安全策略？这些问题决定采用轻量底座的收益能否覆盖后续设计、测试和维护成本。

6.2.2 第二步：接通最小模型链路

安装、版本检查、启动和供应商登录只负责证明模型交互可用。这个阶段应保护凭据，并记录实际使用的版本。视频画面显示的是 pi v0.78.1；供应商数量、交互入口和目录支持可能随版本变化，因此版本是后续复现的重要前提。

²⁴视频来源时间区间：00:30:18.450–00:31:46.510。最终画面从该区间的密集候选帧中选择并裁去桌面背景、人物画面与字幕区域。

6.2.3 第三步：先做一个可验证工具

首个扩展宜选择输入和输出都容易复核的任务。视频中的 `change_context` 读取 Git 变更，并返回文件列表和 diff；验证时既能观察模型是否调用工具，也能把输出与工作区实际变更对照。这个选择让“扩展已加载”和“扩展完成了正确工作”成为两个可分别检查的状态。

6.2.4 第四步：加入生命周期与安全策略

生命周期事件提供观察点，工具调用拦截提供执行前的控制点。视频用危险命令字符串触发确认，并在拒绝后检查目标文件仍然存在，展示了“策略前置、人工确认、明确阻塞、结果复核”的完整形态。生产环境应把简单字符串规则升级为结构化命令解析、动作分类、路径边界、权限策略与审计记录。

6.2.5 第五步：用技能和提示模板固定任务

`change-assessment` 技能把评估方法固定为“改了什么、影响范围、风险点、复核建议”；`/assess-change` 提示命令再把技能、`change_context` 工具和四段输出组织为稳定入口。工具提供数据，技能提供方法，提示模板提供调用合同。三层职责分开，才能分别演进和验证。

6.2.6 第六步：用 pi 扩展包分发并做安装后核验

最后将扩展、技能、提示模板和主题按实际需要放入 pi 扩展包，通过 `package.json` 声明资源目录。团队成员安装后，还要用包列表、资源加载状态和最小任务验证确认运行环境已经采用预期资产。这个步骤把个人配置提升为可追踪的团队交付。

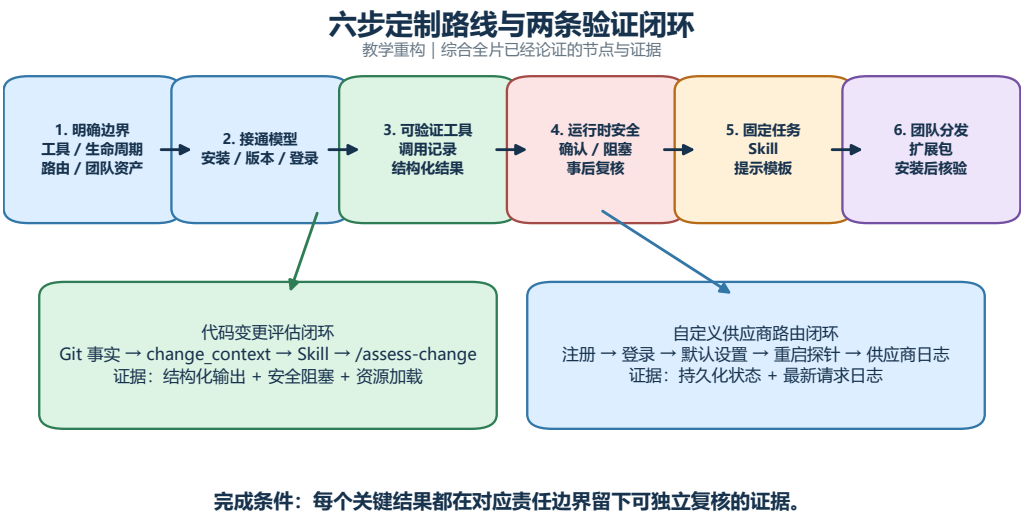


图 28: 六步实施路线与两条验证闭环（教学重构）。上层路线说明能力如何逐层落地；下层闭环把代码变更评估和供应商路由分别绑定到可独立复核的证据。

6.3 两个闭环，共同证明一条工程原则

6.3.1 代码变更评估闭环

代码变更评估链路把不同层的责任串在一起：

1. Git 工作区提供待分析事实；
2. `change_context` 把事实转为结构化上下文；
3. `change-assessment` 技能规定评估方法；
4. `/assess-change` 固定调用顺序和交付格式；
5. 安全扩展在危险动作前介入；
6. pi 扩展包把同一套任务资产交给团队。

这个闭环的证据分别位于工具调用记录、结构化输出、阻塞结果和安装后的资源加载状态。任何一处缺失，都需要回到对应层诊断；例如，命令入口存在却没有结构化 Git 上下文，问题应首先沿提示模板与工具调用关系排查。

6.3.2 自定义供应商路由闭环

自定义供应商案例揭示了另一类常见误判：LiteLLM 出现在供应商列表并完成配置，只能证明注册和认证入口存在；终端生成正常回答，也不能自动证明请求走过 LiteLLM。视频通过下面的证据链完成验证：

1. 扩展注册 LiteLLM；
2. 登录流程保存供应商配置；
3. `settings.json` 持久化默认供应商和默认模型；
4. 重启后发送探针请求；
5. LiteLLM 最新请求日志出现相应提示与响应。

第一次探针没有出现在 LiteLLM 日志中，说明此前持久化的默认状态仍在生效。修改默认设置、重启并再次探测后，供应商侧日志才给出路由证据。这里最重要的教学结论是：注册、认证、默认选择、运行请求和供应商日志属于不同状态，必须用各自的证据检查。

两条闭环共享同一原则：关键结果要在责任边界上被独立观察。安全策略要以阻塞结果和事后状态证明；模型路由要以持久化默认值和供应商日志证明；团队分发要以安装记录和资源加载证明。

6.4 限制、风险与未知项

视频建立了定制方法，也留下了明确的证据边界。

版本漂移 演示画面显示 pi v0.78.1。目录、供应商列表、界面入口和清单字段需要以实际版本复核。

视觉可读性 已验证视频只有 852×480。小字号 JSON 值和部分模型标识无法可靠逐字恢复；精确标识应由清晰画面或直接来源证明。

字幕误识别 唯一字幕是平台 AI 中文字幕，反复误识别 pi agent、LiteLLM、技能、提示模板、主题和 pi 扩展包。正文采用界面、代码和术语合同能够确认的形式。

安全示例的局限 对命令字符串进行简单匹配，只能说明拦截接口如何工作。别名、脚本、间接调用和复杂参数都可能造成误报或漏报。

生产治理缺口 视频没有覆盖依赖锁定、包签名、升级回滚、细粒度权限模型和并发运行。这些能力需要在团队交付前另行设计和验证。

“视频未覆盖”表示当前证据无法支持确定结论。依赖锁定、签名、回滚和权限模型属于后续工程议题，不能根据演示包的成功安装推断为已经具备。

6.5 从个人定制走向可信团队运行

团队化之后，设计重点会从“某项功能能否实现”移动到“谁可以发布、谁可以安装、发生问题怎样恢复”。下面三组开放问题可以作为下一阶段的设计入口。

6.5.1 版本治理与可信发布

团队需要决定包版本怎样标记、依赖怎样锁定、来源怎样验证、变更怎样审查，以及发布后怎样证明每台机器安装了同一版本。若扩展能够执行命令或访问外部服务，还需要对发布者身份、代码审查和制品完整性建立更强约束。

6.5.2 从字符串规则升级为结构化策略

安全拦截需要理解动作类型、目标路径、参数语义和调用来源。可继续追问：哪些动作必须人工确认？哪些目录永远禁止写入？哪些工具只允许读取？阻塞与放行怎样记录？失败后怎样验证系统状态没有发生越权变化？这些问题共同把教学级拦截推进到可审计的权限策略。

6.5.3 多供应商环境中的自动审计

当多个模型供应商并存时，运行系统应能够记录默认选择、实际请求路由和供应商侧证据之间的对应关系。理想的审计记录还要回答请求使用了哪个模型、何时切换、由谁修改设置、失败时是否回退，以及供应商日志能否与本地会话关联。

6.6 最终记忆卡

记忆项	应保留的判断
一句话选择规则	需要掌控扩展边界和团队资产时选择定制底座；需要立即使用完整现成功能时先评估预装型方案。
四层能力栈	扩展/工具负责运行时，技能负责方法，提示模板负责入口与格式，pi 扩展包负责分发；主题横向管理颜色。
代码评估闭环	Git 事实 → change_context → change-assessment → /assess-change → 安全约束与团队分发。
路由验证闭环	注册与认证 → 默认设置 → 重启探针 → 供应商日志。
三项生产化风险	安全策略过于粗糙、包版本与来源缺乏治理、关键运行结果缺少独立审计证据。

6.7 本章小结

pi agent 的核心价值在于提供可组合、可验证、可分发的扩展边界。落地路线应从控制需求出发，依次接通模型、建立可验证工具、加入运行时安全、固化任务方法与入口，最后通过 pi 扩展包完成团队分发。每一步都需要与责任边界相匹配的证据；能够运行只说明链路已经启动，边界清晰、结果可复核、资产可治理才构成完整交付。